

in ``00_dynamic_todo_workspace`` folder, in a directory fullstack-test - build a modern end2end dynamic todo list application industry best experience of ux and nice features - make reasonable assumptions.

Monday 9:50 PM

Worked for 7m 50s

Built a complete full-stack todo application in
[fullstack-test](C:\\Users\\James\\OneDrive\\Desktop\\vscode
files\\CMPE255-assignment\\00_dynamic_todo_workspace\\fullstack-test\\README.md),
following the referenced workspace's client/server direction. ([reference](#))

Features include:

- Persistent JSON-backed REST API
- Create, edit, complete, reopen, and delete tasks
- Priorities, tags, notes, statuses, and due dates
- Today, upcoming, completed, and priority views
- Search and smart sorting
- Progress dashboard
- Responsive mobile navigation
- Light/dark themes
- Accessible dialogs and controls
- **N** and **/** keyboard shortcuts
- Input validation and safe HTML rendering
- Zero external dependencies

Validation completed:

- 3/3 automated tests pass
- Homepage returns HTTP 200
- Create/update/delete API smoke test passes
- The temporary smoke-test task was removed

Run it:

```
cd "00_dynamic_todo_workspace\\fullstack-test"  
npm start
```

Then open <http://localhost:4173>.

Edited 7 files+158-5Review changes

UndoReview

00_dynamic_todo_workspace/fullstack-test/README.md00_dynamic_todo_workspace/fullstack
-test/README.md+21-0

00_dynamic_todo_workspace/fullstack-test/package.json00_dynamic_todo_workspace/fullstack-test/package.json+1-0

00_dynamic_todo_workspace/fullstack-test/public/app.js00_dynamic_todo_workspace/fullstack-test/public/app.js+36-0

Show 4 more files

Monday 9:58 PM

in [`01\\_nyc\\_taxi\\_trip\\_prediction`](#) folder,

Now on to another project - You will do end2end a data science project of including data, training, deployment, crisp-dm framework, and an awesome front end... Kaggle NYC taxi challenge

Monday 10:09 PM

Worked for 7h 53m 51s

Built the complete project in

[01_nyc_taxi_trip_prediction](C:\\Users\\James\\OneDrive\\Desktop\\vscode files\\CMPE255-assignment\\01_nyc_taxi_trip_prediction\\README.md).

Included:

- CRISP-DM lifecycle and model card
- Reproducible 12,000-trip demonstration dataset
- Official Kaggle data downloader
- Data-quality validation and geographic filtering
- Temporal, distance, bearing, and rush-hour features
- Median baseline and gradient-boosted model
- Chronological validation split
- Versioned model artifact and metrics
- FastAPI prediction, metadata, health, and OpenAPI endpoints
- Responsive RideCast NYC frontend
- Route presets, prediction intervals, arrival time, and distance display
- Feature and API tests
- Reproducible EDA script

Demo model results:

- Model RMSLE: 0.0865
- Baseline RMSLE: 0.4859
- MAE: 38.2 seconds
- R²: 0.9803

These are synthetic-sample pipeline results, not claims about real Kaggle performance. Kaggle uses RMSLE as the official competition metric. ([Kaggle competition](#)) The underlying competition data originates from NYC taxi trip records. ([NYC TLC data](#))

The live service successfully returned HTTP 200 for:

- Frontend
- Health endpoint
- Prediction endpoint

Run it:

```
cd "01_nyc_taxi_trip_prediction"
python -m uvicorn app.main:app --reload
```

Open:

<http://127.0.0.1:8000>

To retrain:

```
python scripts/generate_sample_data.py
python -m src.train
```

To use the real competition data:

```
python scripts/download_data.py
python -m src.train --data data/raw/train.csv --max-rows 500000
```

The current environment lacks `pytest`; install the declared requirements before running `python -m pytest -q`.

Edited 18 files+269-3Review changes

UndoReview

01_nyc_taxi_trip_prediction/.gitignore01_nyc_taxi_trip_prediction/.gitignore+8-0

01_nyc_taxi_trip_prediction/README.md01_nyc_taxi_trip_prediction/README.md+49-3

01_nyc_taxi_trip_prediction/app/__init__.py01_nyc_taxi_trip_prediction/app/__init__.py+1-0

Show 15 more files

6:03 AM

in

[`02 nano llm transformer`](#),

build a simple llm and chatbot (with state of art primitives but fit in my laptop gpu) - make sure you follow crisp-dm framework and also include nice data science admin dashboard...

6:05 AM

Worked for 7m 50s

Built the complete project in

[02_nano_llm_transformer](C:\\Users\\James\\OneDrive\\Desktop\\vscode files\\CMPE255-assignment\\CMPE255-assn1\\Part2\\02_nano_llm_transformer\\README.md), following the referenced NanoLlama direction. ([reference](#))

Included:

- Decoder-only PyTorch transformer
- RMSNorm, RoPE, grouped-query attention, Flash-compatible SDPA, and SwiGLU
- Approximately 10M-parameter laptop-friendly defaults
- Mixed precision, gradient accumulation, clipping, warmup, and cosine decay
- Reproducible UTF-8 byte-tokenized dataset pipeline
- Training/validation split with corpus fingerprinting
- Stateful and streaming FastAPI chatbot
- Temperature, top-k, top-p, repetition penalty, and token controls
- Chat Studio, Admin Lab, and architecture visualization
- Loss, perplexity, data-quality, configuration, and runtime telemetry
- CRISP-DM report and data card
- Honest demo mode before a checkpoint exists

Validation completed:

- Corpus preparation succeeded
- 20 documents and 4,169 bytes processed
- SHA-256 provenance recorded
- All Python files compile
- Tokenizer round-trip and configuration checks pass
- Stateful demo chat passes
- Frontend and health endpoint return HTTP 200
- Verification server was shut down cleanly

PyTorch is not installed in the current interpreter, so a neural checkpoint was not trained. Install and train it with:

```
cd "CMPE255-assn1\\Part2\\02_nano_llm_transformer"
python -m pip install -r requirements.txt
python scripts/prepare_data.py
python -m core.train
python -m uvicorn server.main:app --reload --port 8002
```

Then open <http://127.0.0.1:8002>.

For a quick CPU smoke-training run:

```
python -m core.train --max-steps 30 --batch-size 4 --block-size 128 --d-model 128 --layers 2
--heads 4 --kv-heads 2
Edited 20 files+376-1Review changes
UndoReview
CMPE255-assn1/Part2/02_nano_llm_transformer/.gitignoreCMPE255-assn1/Part2/02_nano_llm
_transformer/.gitignore+6-0
CMPE255-assn1/Part2/02_nano_llm_transformer/README.mdCMPE255-assn1/Part2/02_nano
_llm_transformer/README.md+58-0
CMPE255-assn1/Part2/02_nano_llm_transformer/core/__init__.pyCMPE255-assn1/Part2/02_na
no_llm_transformer/core/__init__.py+1-0
Show 17 more files
6:13 AM
```

in Part2/`03\_customer\_segmentation\_clustering`, Now lets do another project - clustering using popular kaggle data set - make sure you follow crisp-dm framework and also include nice data science admin dashboard...

6:15 AM
Worked for 6m 18s

Built the complete project in
[03_customer_segmentation_clustering](C:\\Users\\James\\OneDrive\\Desktop\\vscode
files\\CMPE255-assignment\\CMPE255-assn1\\Part2\\03_customer_segmentation_clustering\\R
EADME.md), following the referenced project structure. ([reference](#))

It includes:

- Kaggle Customer Personality Analysis download workflow
- Reproducible 2,240-customer synthetic dataset
- CRISP-DM report and data card
- Schema validation, missing-value treatment, deduplication, anomaly filtering, and winsorization
- Fifteen engineered behavioral and demographic features
- Robust scaling and K-Means clustering
- Automated comparison of $k=2...8$
- Silhouette, Calinski-Harabasz, Davies-Bouldin, inertia, and stability metrics
- PCA customer landscape visualization
- Business-readable personas and activation recommendations
- Saved model, customer assignments, metrics, and metadata
- FastAPI dashboard and customer-assignment API
- Responsive executive, persona, model-lab, and scoring interfaces

The sample model selected two segments:

- High-Value Loyalists: 51.7%

- Digital Enthusiasts: 48.3%

Sample validation:

- Silhouette: 0.2136
- Stability: 0.9982
- Records retained: 99.8%
- PCA visualization coverage: 46.38%

The generated sample confirms the pipeline; final business conclusions should use the real [Kaggle Customer Personality Analysis dataset](#).

All Python files compile, direct customer assignment succeeds, and the dashboard, health, and telemetry endpoints return HTTP 200.

Run it:

```
cd "CMPE255-assn1\Part2\03_customer_segmentation_clustering"
python -m uvicorn app.main:app --reload --port 8003
```

Open <http://127.0.0.1:8003>.

For real Kaggle data:

```
python scripts/download_data.py
python -m src.train --data data/raw/marketing_campaign.csv
Edited 16 files+205-1Review changes
UndoReview
CMPE255-assn1/Part2/03_customer_segmentation_clustering/.gitignoreCMPE255-assn1/Part2
/03_customer_segmentation_clustering/.gitignore+6-0
CMPE255-assn1/Part2/03_customer_segmentation_clustering/README.mdCMPE255-assn1/P
art2/03_customer_segmentation_clustering/README.md+43-0
CMPE255-assn1/Part2/03_customer_segmentation_clustering/requirements.txtCMPE255-assn
1/Part2/03_customer_segmentation_clustering/requirements.txt+8-0
Show 13 more files
6:21 AM
```

in Part2/[04_associative_pattern_mining](#),

Now lets do another project - associative pattern mining using popular kaggle data set - make sure you follow crisp-dm framework and also include nice data science admin dashboard...

6:22 AM

Worked for 6m 30s

Built the complete project in

[04_associative_pattern_mining](C:\\Users\\James\\OneDrive\\Desktop\\vscode

files\\CMPE255-assignment\\CMPE255-assn1\\Part2\\04_associative_pattern_mining\\README.md).

It uses the popular Kaggle Online Retail dataset schema, containing invoices, products, quantities, prices, dates, customers, and countries. ([Kaggle dataset](#))

Included:

- Complete CRISP-DM report and data card
- Kaggle downloader
- Reproducible synthetic retail dataset
- Cancellation, return, duplicate, missing-value, and service-line cleaning
- Dependency-light Apriori implementation
- Frequent singles, pairs, and triples
- Support, confidence, lift, leverage, and conviction
- Chronological 80/20 validation
- Holdout confidence, drift, and rule-stability checks
- Deployable JSON rule artifact
- Basket recommendation API
- Interactive product-association network
- Searchable rule explorer
- Basket recommendation studio
- Product and geographic performance views
- Data-quality and model-governance dashboard

Sample pipeline results:

- 20,171 raw invoice lines
- 5,000 transactions
- 24 products
- 4.02 average basket size
- 84 frequent itemsets
- 216 qualified rules
- 216 sample holdout-stable rules
- 1,000 chronological holdout baskets

Direct recommendation validation produced:

RED RETROSPOT GIFT WRAP
→ JUMBO BAG RED RETROSPOT

Confidence: 75.35%

Lift: 4.14×

Support: 13.45%

All Python files compile, direct rule and recommendation checks pass, and the dashboard, health, and artifact endpoints return HTTP 200.

Run it:

```
cd "CMPE255-assn1\Part2\04_associative_pattern_mining"  
python -m uvicorn app.main:app --reload --port 8004
```

Open <http://127.0.0.1:8004>.

For the real Kaggle dataset:

```
python scripts/download_data.py  
python -m src.mine --data "data/raw/Online Retail.csv"  
Edited 16 files+205-1Review changes  
UndoReview  
CMPE255-assn1/Part2/04_associative_pattern_mining/.gitignoreCMPE255-assn1/Part2/04_ass  
ociative_pattern_mining/.gitignore+6-0  
CMPE255-assn1/Part2/04_associative_pattern_mining/README.mdCMPE255-assn1/Part2/04  
_associative_pattern_mining/README.md+36-0  
CMPE255-assn1/Part2/04_associative_pattern_mining/requirements.txtCMPE255-assn1/Part2/  
04_associative_pattern_mining/requirements.txt+6-0  
Show 13 more files  
6:29 AM
```